

Initialization, Normalization, and Debugging

Scaler School of Technology | Deep Learning I | Week 3 — Session 2/2 | Post-Lecture Review & Practice



Key Takeaways

1 Initialization Determines Training Success

Gradients are products of many factors (chain rule). If weights are too small or too large, signals vanish or explode exponentially. Xavier init (for tanh/sigmoid) and He init (for ReLU) set weight variance so activations stay stable across all layers.

2 Normalization Stabilizes Training

BatchNorm normalizes activations across the batch (per feature) — great for CNNs with large batches. LayerNorm normalizes across features (per sample) — essential for Transformers and variable-length sequences. Always remember `model.eval()` for BN at inference!

3 Debug Systematically, Not Randomly

Follow the checklist: (1) overfit one batch, (2) check shapes, (3) verify loss at init $\approx \ln(k)$, (4) gradient check, (5) train on small subset, (6) scale up. Gradient checking compares analytical gradients against numerical approximation to catch backprop bugs.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** Why does Xavier initialization fail when used with ReLU activations? What does He initialization do differently?
- Q2.** During inference, where does BatchNorm get its mean and variance from? What happens if you forget `model.eval()`?
- Q3.** You're training a 10-class classifier. The initial loss is 4.5 instead of ≈ 2.3 . What might be wrong?
- Q4.** Explain the difference between what BatchNorm and LayerNorm normalize over. Why does this matter for Transformers?
- Q5.** Your model's loss goes to NaN after 3 epochs. List 3 potential causes and fixes.

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Initialization Showdown

Build a 20-layer MLP (hidden size 256, ReLU activations) and pass a random batch through it **without training**. Record the standard deviation of activations at each layer for three init strategies: (a) `Normal(0, 0.01)`, (b) Xavier, (c) He. Plot all three on the same graph. Which stays flat? Which collapses?

Exercise 2: BatchNorm from Scratch

Implement `batch_norm_forward(x, gamma, beta)` that computes BN manually (no `nn.BatchNorm1d`). Test with a (32, 64) tensor. Verify your output matches PyTorch's `nn.BatchNorm1d` to within 10^{-5} . Then train a 5-layer MLP on MNIST with and without your BN layer — compare convergence speed (plot loss curves for 10 epochs).

Exercise 3: The Debugging Checklist in Practice

The following model has 3 bugs. Find and fix all of them using the debugging checklist: `model = nn.Sequential(nn.Linear(784, 256), nn.BatchNorm1d(256), nn.ReLU(), nn.Linear(256, 10), nn.ReLU())`. Train on MNIST. Apply each checklist step: (1) Does it overfit 1 batch? (2) Is initial loss $\approx \ln(10)$? (3) Do all layers have gradients?

Challenge Problem

The Normalization Ablation on a Deep Network

Build a 30-layer MLP (hidden size 128, ReLU) and train on Fashion-MNIST. Compare these 5 configurations:

1. No normalization, random init (`Normal(0, 0.01)`)
2. No normalization, He init
3. He init + BatchNorm after every layer
4. He init + LayerNorm after every layer
5. He init + BatchNorm, but **forget** `model.eval()` at test time

Expected output: A table showing train accuracy, val accuracy, and whether training converged for each. Plot gradient norms of the first layer across training for configs 1–4.

Bonus: For config 5, run inference 20 times on the same batch and measure variance of predictions. Why is it non-zero?

- **Xavier Initialization** — Glorot & Bengio, 2010, “*Understanding the Difficulty of Training Deep Feedforward Neural Networks*.” The paper that made deep networks trainable. Short and readable.
- **He Initialization** — He et al., 2015, “*Delving Deep into Rectifiers*.” Extends Xavier for ReLU. Also introduces PReLU.
- **Batch Normalization** — Ioffe & Szegedy, 2015, “*Batch Normalization: Accelerating Deep Network Training*.” One of the most cited DL papers. Read Sections 1–3.
- **Layer Normalization** — Ba et al., 2016, “*Layer Normalization*.” Why BN fails for RNNs and how LN fixes it. Became essential with Transformers.

Online Resources & Tools

- **PyTorch nn.init** — All initialization functions with formulas and usage
- **PyTorch BatchNorm1d/2d** — Docs explaining train vs eval mode behavior
- **CS231n: Neural Net Tips** — Stanford’s practical guide to debugging and training
- **Weights & Biases Debugging Guide** — Common training failures with visual examples
- **torch.autograd.gradcheck** — Built-in gradient verification utility
- **Karpathy’s “Recipe for Training NNs”** — Blog post with the debugging checklist philosophy

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about the chain rule and gradient products?** Revisit Session 3 (Backpropagation) notes. The key insight: gradients through n layers = product of n partial derivatives. If each is < 1 , the product vanishes.
- **Fuzzy on variance and standard deviation?** Khan Academy’s “Variance and Standard Deviation” module (15 min). You need this to understand why $\text{Var}(w) = 2/n$ keeps things stable.
- **Not sure how `model.train()` vs `model.eval()` works?** PyTorch tutorial “What is torch.nn really?” explains module modes. Critical for BatchNorm and Dropout behavior.
- **Confused about what “normalize” means here?** It’s the same z-score from statistics: $(x - \mu)/\sigma$. BN and LN just differ in *which dimension* they compute μ and σ over.

Answers — Test Your Understanding

- A1.** Xavier assumes a symmetric activation (derivative ≈ 1 around 0). ReLU kills half the values (all negatives become 0), halving the variance each layer. He init doubles the variance ($2/n_{\text{in}}$ vs $1/n_{\text{in}}$) to compensate.
- A2.** From **running averages** accumulated during training. Without `model.eval()`, BN uses the current batch’s statistics at test time — predictions become batch-dependent and noisy.
- A3.** Expected initial loss for 10-class = $\ln(10) \approx 2.3$ (random uniform predictions). Loss of 4.5 suggests: wrong loss function, mislabeled data, or architecture bug (e.g., applying softmax twice).
- A4.** BN normalizes each **feature across the batch** (needs multiple samples). LN normalizes each **sample across features** (self-contained). Transformers use variable-length sequences and often `batch_size=1` at inference — BN would break.
- A5.** (1) Exploding gradients $\rightarrow$ lower LR or add gradient clipping. (2) Bad initialization $\rightarrow$ use He init. (3) Numerical overflow in loss (e.g., $\log(0)$) $\rightarrow$ add epsilon or use numerically stable implementations.